Операционные системы

Кондратьев Александр

Virtualizing the CPU

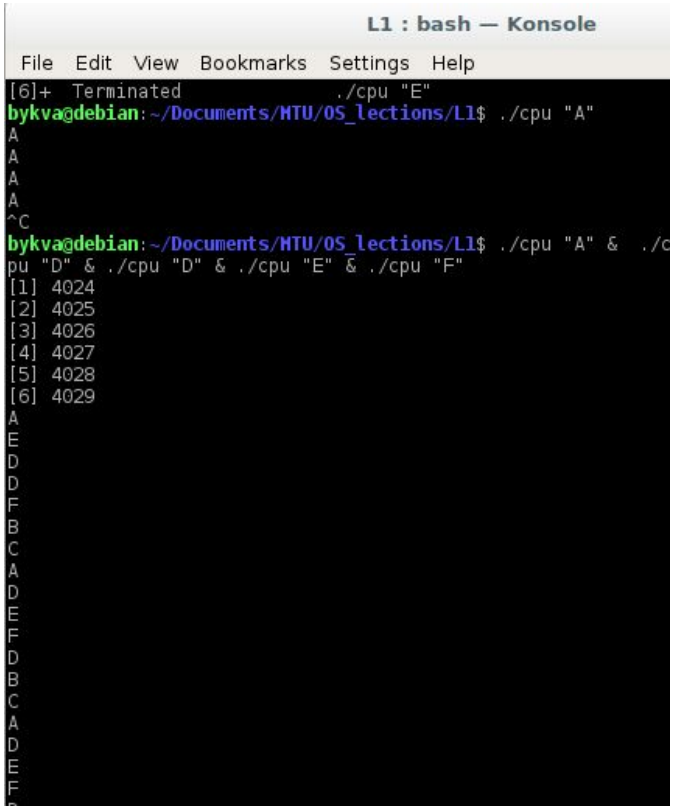
```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>
#include <assert.h>
#include "common.h"

int main(int argc, char *argv[])
{
    if (argc != 2) {
        fprintf(stderr, "usage: cpu <string>\n");
        exit(1);
    }
    char *str = argv[1];

    while (1) {
        Spin(1);
        printf("%s\n", str);
    }
    return 0;
}
```

https://www.youtube.com/watch?v=zDwT5fUcki4

Virtualizing the CPU

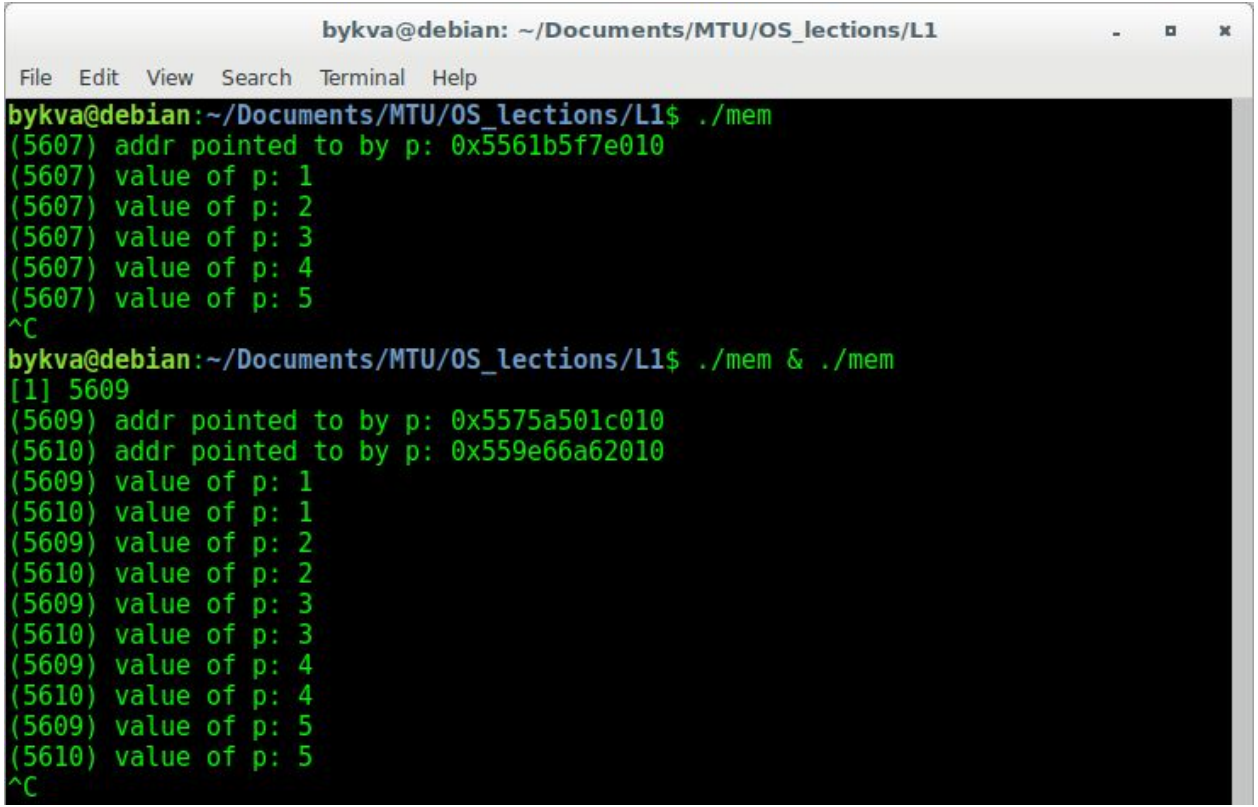


Virtualizing Memory

```c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include "common.h"

int main(int argc, char *argv[]) {
    int *p = malloc(sizeof(int));
    assert(p != NULL);
    printf("(%d) addr pointed to by p: %p\n", (int) getpid(), p);
    *p = 0;
    while (1) {
  Spin(1);
  *p = *p + 1;
  printf("(%d) value of p: %d\n", getpid(), *p);
    }
    return 0;
}
```

Virtualizing Memory



Virtualizing Memory

```
Running two copies of programm (ASLR disabled)
./mem & ./mem &
(6361) addr pointed to by p: 0x555555756010
(6362) addr pointed to by p: 0x555555756010
(6361) value of p: 1
(6362) value of p: 1
(6361) value of p: 2
(6362) value of p: 2
(6361) value of p: 3
(6362) value of p: 3
./mem.sh: line 20:  6361 Terminated              ./mem
./mem.sh: line 20:  6362 Terminated              ./mem
```

https://youtu.be/jnlKRnoT1m0

Concurrency

```c
#include <stdio.h>
#include <stdlib.h>
#include "common.h"
#include "common_threads.h"

volatile int counter = 0;
int loops;

void *worker(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
	counter++;
    }
    return NULL;
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
	fprintf(stderr, "usage: threads <loops>\n");
	exit(1);
    }
    loops = atoi(argv[1]);
    pthread_t p1, p2;
    printf("Initial value : %d\n", counter);
    Pthread_create(&p1, NULL, worker, NULL);
    Pthread_create(&p2, NULL, worker, NULL);
    Pthread_join(p1, NULL);
    Pthread_join(p2, NULL);
    printf("Final value   : %d\n", counter);
    return 0;
}
```

Concurrency

